

DEPARTMENT OF CSE

COURSE CODE : CSE 402

COURSE TITLE : Operating System Lab

REPORT : 06

REPORT NAME : FCFS & SSTF Disk Scheduling

SUBMITTED BY:

NAME : SUJANA MAHMUDA NITE

STUDENT ID. : 23101120

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C-1

SUBMITTED TO:

ATIA RAHMAN ORTHI

UAP

DATE OF SUBMISSION: 12-10-2026

Problem Statement

Implement FCFS (First Come First Serve) and SSTF (Shortest Seek Time First) disk scheduling algorithms for the given disk request queue. Determine the order in which the requests are served, calculate the total seek time for both algorithms, and compare their performance to identify which algorithm is more efficient for the given data.

Given Data

Item	Value
Request Queue	[0, 30, 41, 62, 14]
Initial Head Position	20

Objective

The objective of this experiment is to understand how FCFS and SSTF disk scheduling algorithms move the disk head to serve pending requests. The lab also compares the total head movement (seek time) produced by the two methods and explains why one method performs better for the given request sequence.

Working Principle

FCFS Disk Scheduling: The First-Come, First-Served (FCFS) algorithm processes disk requests according to their arrival order. The disk head first moves to the earliest request in the queue and then visits each remaining request sequentially until every request has been served.

SSTF Disk Scheduling: The Shortest Seek Time First (SSTF) algorithm chooses the pending disk request that requires the least movement from the current head position. After completing each request, the disk head checks the remaining requests and selects the nearest one. This procedure repeats until all requests have been processed.

Seek Time Calculation

The total head movement is determined by calculating the distance traveled by the disk head between consecutive positions. For each movement, the absolute difference between the current head position and the selected request position is calculated.

Formula:

Seek Distance = |Current Head Position − Next Request Position|

Total Head Movement = Sum of all individual seek distances.

FCFS Calculation

Movement	Seek Distance
20 -> 0	20
0 -> 30	30
30 -> 41	11
41 -> 62	21
62 -> 14	48
Total	**130**

FCFS Order: [0, 30, 41, 62, 14]

FCFS Total Seek Time = 130

SSTF Calculation

Movement	Seek Distance
20 -> 14	6
14 -> 0	14
0 -> 30	30
30 -> 41	11
41 -> 62	21
Total	**82**

SSTF Order: [14, 0, 30, 41, 62]

SSTF Total Seek Time = 82

Advantages and Disadvantages of FCFS

Advantages:

- Easy to implement because requests are processed in their arrival sequence.
- Provides equal treatment by following the original request queue.
- Prevents starvation since requests are handled one after another.

Disadvantages:

- Large differences between consecutive request positions can cause excessive head movement.
- Higher seek time may reduce overall disk performance.
- The algorithm does not select requests based on their distance from the current head position.

Advantages and Disadvantages of SSTF

Advantages:

- Chooses the closest pending request, helping to minimize disk-head movement.
- Generally achieves a lower average seek time than FCFS.
- Can improve disk scheduling efficiency by reducing unnecessary movement between tracks.

Disadvantages:

- Requests located far from the current head position may experience long delays or starvation.
- Finding the nearest request requires additional comparisons during scheduling.
- Request processing does not follow arrival order, making it less fair than FCFS in some situations.

Why SSTF Performs Better in This Experiment

For the given disk-request sequence, SSTF produces a total head movement of 82 units, whereas FCFS requires 130 units. Thus, SSTF reduces the total movement by 48 units, representing approximately a 36.9% reduction compared with FCFS.

This improvement occurs because SSTF prioritizes the nearest pending request instead of following the original queue order. As a result, the disk head travels a shorter overall distance for the given input. Therefore, SSTF demonstrates better performance than FCFS in this particular experiment.

Comparison between FCFS and SSTF

Point	**FCFS**	**SSTF**
Selection Rule	Follows request queue order	Chooses nearest pending request
Initial Head Position	20	20
Service Order	0, 30, 41, 62, 14	14, 0, 30, 41, 62
Total Seek Time	130	82
Starvation	No	Possible for distant requests
Fairness	High	Lower than FCFS
Implementation	Very simple	More decision making
Performance in This Lab	Higher seek time	Lower seek time
Better for Given Data	No	Yes

Source Code

```
req  = [0, 30, 41, 62, 14, 20]
head = 20


req = [r for r in req if r != head]

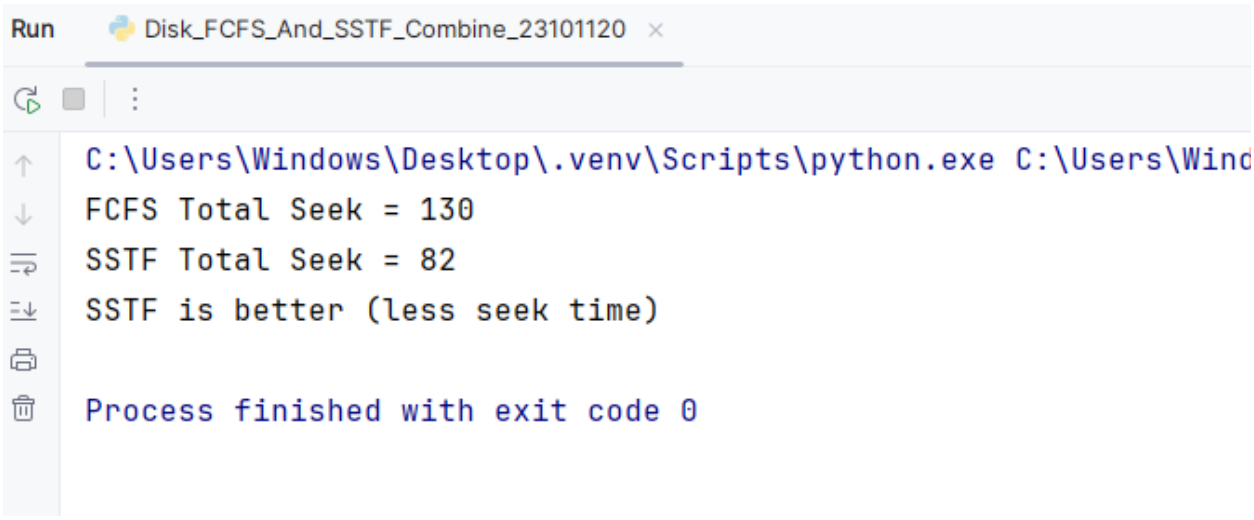

# FCFS
total_fcfs = 0
cur = head
for r in req:
    total_fcfs += abs(r - cur)
    cur = r
```

```
# SSTF
remaining = req.copy()
total_sstf = 0
cur = head
while remaining:
    closest = min(remaining, key=lambda x: abs(x - cur))
    total_sstf += abs(closest - cur)
    cur = closest
    remaining.remove(closest)

print("FCFS Total Seek =", total_fcfs)
print("SSTF Total Seek =", total_sstf)

if total_sstf < total_fcfs:
    print("SSTF is better (less seek time)")
else:
    print("FCFS is better (less seek time)")
```

Output



Discussion

In this experiment, FCFS and SSTF disk scheduling algorithms were evaluated using an identical request queue and initial disk-head position. The FCFS algorithm processed requests in their given sequence, resulting in a total head movement of 130 units. In contrast, SSTF prioritized the closest available request at each step and reduced the total head movement to 82 units.

The comparison demonstrates that the selection strategy has a significant impact on disk-head movement and seek performance. SSTF achieved better results for the given dataset by reducing unnecessary movement. However, it may cause longer waiting times for distant requests, whereas FCFS ensures that requests are handled in arrival order and is easier to implement.

Conclusion

The experiment successfully demonstrated the operation and performance differences between FCFS and SSTF disk scheduling algorithms. Based on the given request sequence, SSTF performed more efficiently, requiring only 82 units of head movement compared with 130 units for FCFS. This result confirms that selecting nearby requests can reduce disk-head movement for the tested dataset. Nevertheless, FCFS offers a simpler and more predictable service order, while SSTF provides better movement efficiency but may lead to starvation for some requests.

GitHub Link:

https://github.com/sujana120/Operating-System-Lab